

Kubernetes 企業實戰教育訓練

備份、復原與 災難復原演練

從 Application Backup 開始，逐步理解資源、PV 資料、etcd 與整座叢集的復原

主講人：李旻恩



Opc Ua Extender

opc UA



SYSTEM



USB UltraLock



IoT Studio



八個單元 + 補充

#	單元	核心問題	實機操作
1	Kubernetes 備份與復原概念	「備份 Kubernetes」到底要備份什麼？	觀念 + 唯讀觀察
2	建立外部 Backup Storage	備份為什麼一定要放叢集外？怎麼確認真的寫出去？	SeaweedFS + BSL
3	第一次 Backup	怎麼判斷備份「真的完成」？先看 YAML 再用 UI 建立	YAML + UI 實測
4	第一次 Restore	怎麼證明資料真的救回來，而不是只有 Pod Running？	YAML + UI 實測
5	etcd 備份與還原策略	etcd snapshot 跟 Velero 差在哪？什麼時候用哪個？	yaml + 講師操作
6	Control Plane Disaster	API Server 起不來時怎麼從 snapshot 救？	runbook
7	Cluster Disaster	叢集整個不見，要照什麼順序重建？	runbook
8	每日備份與 Restore 驗證	怎麼讓備份每天跑、失敗會通知、定期驗證能還原？	YAML + UI 操作
補充	補充：Backup 的陷阱	備份顯示 Completed，為什麼還是救不回來？	課後自學 · 5 個情境

01

Kubernetes 備份與復原概念

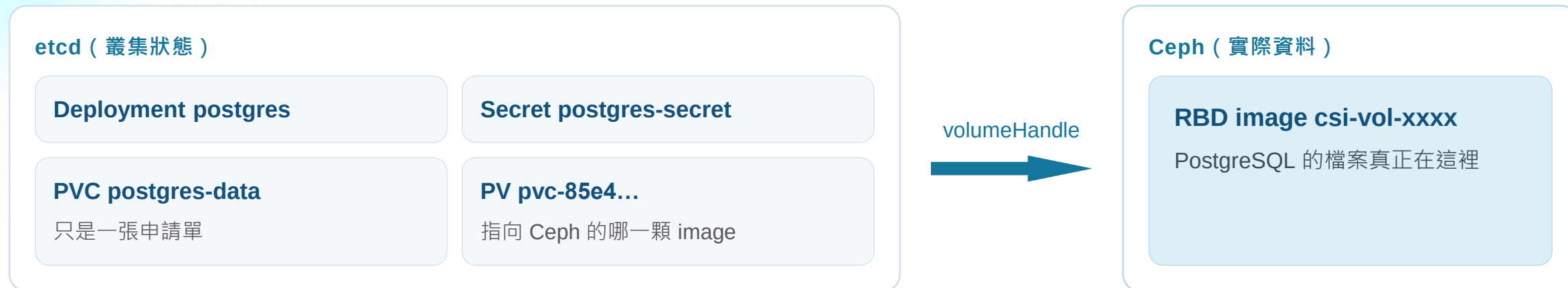
單元 1 先把「備份 Kubernetes」拆開：要備份什麼、住在哪、用什麼救。



Kubernetes Resource 與 Application Data

	Kubernetes Resource (叢集狀態)	Application Data (應用資料)
例子	Deployment、Service、Secret、ConfigMap、PVC 物件、HTTPRoute	PostgreSQL 的資料表、CephFS 上的 dr-test.txt
長什麼樣子	YAML / JSON，幾 KB	資料庫檔案、使用者上傳的檔案，可能數百 GB
住在哪裡	etcd	PV 背後的儲存系統：Ceph RBD / CephFS
誰寫進去的	管理者 kubectl apply、Controller	應用程式在執行期間寫入
能不能重新產生	通常可以：Git 裡有 yaml，GitOps 重新套用	不行。訂單、使用者資料丟了就是丟了
備份工具	Velero (資源)、etcd snapshot (整個叢集狀態)	Velero Volume 備份、應用自己的 dump

etcd 裡只有描述，沒有內容



一句話

etcd 裡只有「這裡應該有一個 10Gi 的 PVC」這個描述。只備份 etcd 或 YAML，救回來的是一個空殼。課程最後的補充單元（情境 1）會實際重現「Pod Running、table 也在，但訂單全不見」。

備份放在哪裡，決定它能活過哪種災難

存放位置	Pod 掛了	Namespace 被刪	Ceph 壞掉	整座叢集消失
只有 CSI Snapshot (存在 Ceph 裡)	✓ 可以	視 deletionPolicy	X 不行	X 不行
PVC 裡另外放一份 dump	✓ 可以	X 不行	X 不行	X 不行
叢集外的 Object Storage (S3)	✓ 可以	✓ 可以	✓ 可以	✓ 可以

角色 1

故障隔離

跟叢集不同機器、不同磁碟 (本課：
10.90.1.125 的 /dev/sdb1)

角色 2

共同語言

Velero、etcd 腳本、資料庫 dump 都會講 S3
API，一套儲存收全部種類的備份

角色 3

可攜性

新叢集只要指向同一個 bucket，就能看到舊叢
集的所有備份 (單元 7 的關鍵)

02

建立外部 Backup Storage

單元 2 備份的第一條規則：備份不能跟被備份的東西一起死掉。



在叢集裡開一個 MinIO 不就好了？

災難	備份在叢集內 Ceph RGW / MinIO	備份在叢集外 SeaweedFS
誤刪 namespace	✓ 救得回來	✓ 救得回來
Ceph 故障 (OSD 大量損壞)	X 備份跟資料一起壞	✓ 沒影響
etcd 損壞、API Server 起不來	⚠ 資料可能還在，但 MinIO / RGW 也靠叢集才能跑	✓ 沒影響
整座叢集重灌 / 機房失火	X 一起消失	✓ 若在另一個機房更好
勒索軟體拿到 cluster-admin	X 一個 kubectl delete 就清光	⚠ 視 S3 帳號權限而定

這座叢集真的有 `rook-ceph-retain-bucket` StorageClass，早期測試備份 `test-ceph-rgw-final` 就是這樣做的。

Backup Storage 的三個原則

原則 1

不同的故障域

不同機器、不同磁碟，最好不同機房

3-2-1：3 份資料、2 種媒介、1 份異地

原則 2

不同的權限域

叢集的 cluster-admin 不應該能刪掉備份

Velero 的 S3 帳號只給需要的權限

bucket 由另一個管理者帳號建立

原則 3

不依賴被保護的系統

Backup Storage 自己要能獨立開機、獨立登入、獨立驗證

03

第一次 Backup：備份 Application

單元 3

先看懂 Backup 的 YAML，再用 Velero UI 實際建立，並學會回答：這份備份真的完成了嗎？



Volume Data Backup 的三種方式

方式	設定	資料放哪	叢集消失還在嗎	備註
1. CSI Snapshot (只拍快照)	snapshotVolumes: true	Ceph 裡	X 不在	最快，但跟資料同一個故障域
2. CSI Snapshot + Data Mover	+ snapshotMoveData: true	S3 (kopia)	✓ 還在	先拍快照取得時間點，再由 node-agent 上傳
3. File System Backup	defaultVolumesToFsBackup: true	S3 (kopia)	✓ 還在	直接讀 Pod 掛載目錄；不需 CSI，但不是時間點快照

這座叢集用**第 2 種**：兼顧時間點一致性與故障域隔離。

Velero Backup 流程

步驟 1

Velero 建立 Backup

根據需求建立一份 Backup 用的 YAML 檔，並套用套叢集中

步驟 2

CSI Snapshot

由 Ceph 儲存系統執行，存在本地 Kubernetes 叢集裡的 Ceph 中

步驟 3

Data Mover

負責把 CSI Snapshot 的 Volume 資料搬到 Backup Storage

步驟 4

Kopia

負責實際的資料內容管理，例如切 chunk、去重、壓縮/加密與 Repository 管理

步驟 5

外部 Backup Storage (S3)

最終保存 Kopia Repository 資料

先看 YAML：一份 Backup 長什麼樣子

backup-dr-demo.yaml

```
apiVersion: velero.io/v1
kind: Backup
metadata:
  name: dr-demo-backup-01
  namespace: velero      # Backup 一律建在 velero ns
spec:
  includedNamespaces:
    - dr-demo            # 只備份 dr-demo
  # excludedResources: [] # 例：排除 events
  # labelSelector: {}    # 注意：套用在每一個資源上
  snapshotVolumes: true  # 對 PVC 做 CSI 快照
  snapshotMoveData: true # 快照內容搬到 S3 (Data Mover)
  defaultVolumesToFsBackup: false
  storageLocation: default # 單元 2 建立的 BSL
  ttl: 720h0m0s           # 保留 30 天後自動刪除
  csiSnapshotTimeout: 10m0s
  itemOperationTimeout: 4h0m0s
```

範圍：includedNamespaces

只備份 dr-demo；跟它有關的 PV 等 cluster 資源會自動帶上。不寫 = 全部 namespace。

Volume：snapshotMoveData

先拍 CSI 快照固定時間點，再由 node-agent 用 kopia 上傳到叢集外的 S3。關掉的話，快照只留在 Ceph 裡。

存放與保留：storageLocation / ttl

寫到單元 2 建立的 BSL (SeaweedFS)；30 天到期，Velero 自動刪除 CR 與 S3 上的檔案。

UI = CLI = YAML UI 精靈送出後，也是在 velero namespace 建立這個 Backup 物件；用 `kubectl -n velero get backup <名稱> -o yaml` 就能對照 UI 實際送出的內容。

備份完成檢查清單：缺一不可

#	檢查	指令 / 位置	本次結果
1	Phase 是 Completed	velero backup get	✓
2	Errors = 0 · Warnings 已逐一看過	velero backup describe	✓ 0 / 0
3	Items backed up = Total items	describe	✓ 19 / 19
4	範圍正確：有 Deployment、Secret、Service、PVC	describe --details	✓
5	每個 PVC 都在 Backup Volumes · Result = succeeded	describe --details	✓ 2 / 2
6	Moved data Size 合理 (不是 0、跟實際資料量同級)	describe / DataUpload	✓ 47.8MB / 45B
7	檔案真的在叢集外	單元 2 的 s3-check	✓
8	能還原，而且資料對	單元 4	→ 下一單元

第 8 項是唯一真正能證明備份有用的一項。補充單元會示範 1~7 全部打勾、第 8 項卻失敗。

```
# 檢查備份訊息，環境中須有安裝 Velero CLI，這邊有提供 GUI 可以檢視  
velero describe backups -n velero dr-demo-backup-01 --details
```

04

第一次 Restore：把 Application 救回來

單元 4

還原成功的定義不是 Pod Running，而是災難前寫進去的那筆資料，現在查得到。



兩種還原方式

	原地還原	還原到新 namespace (namespaceMapping)
前提	原 namespace 已不存在 (或資源已刪除)	原 namespace 可以還活著
風險	原應用其實還在時，同名資源會被跳過 (預設不覆蓋)，容易誤判	幾乎沒有：新的 PV、新的 namespace，完全不碰原應用
用途	真正的災難復原	還原演練、比對資料、救回單筆誤刪資料
本課程	講師 demo：確認有 Completed 備份 → 刪除整個 dr-demo (PV 一併刪除) → 原地還原	實測 (2026-09-23)，每位學員都能安全練習

上課順序建議

先用「還原到新 namespace」讓每位學員安全練習，最後由講師 demo 一次「刪掉 dr-demo → 原地還原」的完整災難流程。

先看 YAML：兩種 Restore 的差別只在幾行

原地還原 (真正的災難) · restore-dr-demo.yaml

```
apiVersion: velero.io/v1
kind: Restore
metadata:
  name: dr-demo-restore-01
  namespace: velero
spec:
  backupName: dr-demo-backup-01 # 從哪一份備份
  includedNamespaces:
    - dr-demo
  restorePVs: true # 還原 PV 資料
  existingResourcePolicy: none # 同名資源：跳過
  itemOperationTimeout: 4h0m0s
```

還原到新 namespace (演練) · restore-to-new-namespace.yaml

```
apiVersion: velero.io/v1
kind: Restore
metadata:
  name: dr-course-restore-01
  namespace: velero
spec:
  backupName: dr-demo-backup-01
  namespaceMapping:
    dr-demo: dr-demo-restore # 改寫 namespace
  restorePVs: true # 建立全新的 PV
```

backupName

必須是 Completed 的備份；還原任務名稱可以自己取，跟備份名稱無關。

restorePVs: true

從 S3 下載 PV 資料到新的 PV。
false 只會建出空的 PVC，資料救不回來。

existingResourcePolicy

none (預設) = 同名資源跳過。原應用還在時看起來成功，其實什麼都沒動。

namespaceMapping

dr-demo → dr-demo-restore：新的 namespace、新的 PV，完全不碰原本的應用。

驗證資料庫資料

```
$ kubectl -n dr-demo-restore exec deploy/postgres -- \
    psql -U drtest -d drdemo -c "SELECT * FROM disaster_test;"
id | test_key | test_value | created_at
---+-----+-----+-----
1 | DR-TEST-DB-001 | Kubernetes Disaster Recovery Test | 2026-09-09 03:36:10.139213
(1 row)
```

2026-09-09

`created_at` 比這次備份 (09-23) 早兩週

這證明了什麼

這是「災難前就存在的資料被救回來」，不是還原後重新產生的。驗證要找**不可能自動重生的資料**。

真的還原成功的判斷標準

Restore Completed

Velero 說它做完了

Pod Running

K8s 說容器起來了

PVC Bound

儲存說磁碟接上了

資料查得到

只有這一層證明「救回來了」

應用功能正常

登入、讀寫、對外連線：業務方確認

每往下一層，需要的驗證知識就越接近應用本身。還原演練一定要有應用負責人參與。

05

建立 etcd 備份與還原策略

單元 5 Velero 保護的是應用；etcd snapshot 保護的是叢集本身。



Snapshot 只有一半

snapshot 以外必須備份的檔案	少了它會怎樣
/etc/kubernetes/encryption-config.yaml (+ encryption/)	API Server 讀不出任何 Secret (Internal error: failed to decrypt)
/etc/kubernetes/pki/ca.* 、 sa.*	所有 kubeconfig 、 ServiceAccount token 失效 ; kubelet 連不上
/etc/kubernetes/pki/etcd/*	etcd 成員之間、API Server 到 etcd 的 mTLS 無法建立
/etc/kubernetes/manifests/*.yaml	不知道原本的 API Server / etcd 啟動參數 (audit 、 OIDC 、 加密...)

```
# 在每台 control-plane 上以 root : 上面四類檔案 → tar 打包 → 用另外保存的密碼以 AES-256 加密 → 複製到叢集外
tar czf - /etc/kubernetes/{pki,manifests,encryption,encryption-config.yaml} \
    | openssl enc -aes-256-cbc -pbkdf2 -pass file:<密碼檔> -out cp-files-<node>.enc
```

etcd Backup 與 Velero Backup 的差異

	etcd snapshot	Velero Backup
備份什麼	etcd 裡的全部 key (整座叢集狀態)	選定範圍的 K8s 資源 + PV 資料
怎麼取得	直接連 etcd (需要 etcd 憑證)	透過 API Server (需要 RBAC 權限)
PV 資料	X 沒有	✓ CSI Snapshot + Data Mover
粒度	全有或全無	namespace / 資源種類 / label
一致性	整座叢集同一時間點 (原子性)	逐一讀取資源，非原子性
還原影響	整座叢集回到過去，所有租戶受影響	只影響被還原的範圍
需要叢集活著嗎	不需要 (就是用來救死掉的叢集)	需要 (API Server 要能用)
能還原到別的叢集嗎	X 不建議 (綁 Node、IP、憑證)	✓ 設計目標之一

它們互補，不是二選一。

決策樹：什麼情況用哪一種 Restore



06

Control Plane Disaster : etcd 還原

單元 6 · runbook

API Server 起不來時，怎麼從 snapshot 把整座叢集救回來。



還原流程：每一步 3 台都完成，才進下一步



為什麼一定要同步

任何一台還在用舊資料目錄跑 etcd，就可能跟新成員互相加入，組出分裂的叢集。

07

Cluster Disaster : 整個叢集消失

單元 7 · runbook

叢集整個不見，要照什麼順序重建？



DR 保險箱：災難前就必須存在叢集外

❑ **velero-repo-credentials**

kopia repository 密碼

❑ **Velero S3 credentials**

credentials-velero

❑ **etcd 備份 S3 credentials**

另一組只能存取 etcd-backups 的帳號

❑ **TLS 憑證與私鑰、Root CA**

gateway-tls 與自簽 CA

❑ **各應用的外部相依資訊**

外部 DB、SMTP、OAuth client secret...

❑ **本 repo**

manifest、Helm values、Velero 安裝參數

❑ **最新的叢集基準紀錄**

節點、PV、SC、Gateway、Ceph 狀態

❑ **runbook 的紙本 / 離線版本**

叢集和 Wiki 都掛了時還拿得到

重建順序總覽



原則

基礎元件用它自己的安裝方式重建，應用才用 **Velero** 還原。用 Velero 還原 kube-system、rook-ceph、velero，會把舊叢集的節點名稱、IP、憑證一起倒回新叢集，造成難以除錯的問題。

Step 6：重新連接外部 Backup Storage

```
# 1. 先建立 namespace 與「舊的」repo 密碼 Secret ( 從 DR 保險箱取出 )
kubectl create namespace velero
kubectl apply -f velero-repo-credentials.yaml      # 災難前匯出的那份
# 平時就要匯出 : kubectl -n velero get secret velero-repo-credentials -o yaml \
# | grep -vE 'uid|resourceVersion|creationTimestamp' > velero-repo-credentials.yaml

# 2. 用跟舊叢集相同的參數安裝 Velero，指向同一個 bucket ( s3Url http://10.90.1.125:8333 )
velero install --provider aws --bucket velero --secret-file ./credentials-velero --use-node-agent --uploader-type kopia ...

# 3. 建議先把 BSL 設成唯讀，避免新叢集誤刪 / 誤寫舊備份 ( TTL 清理也會暫停 )
kubectl -n velero patch bsl default --type merge -p '{"spec":{"accessMode":"ReadOnly"}}'

# 4. 確認舊備份自動同步出現
velero backup-location get      # Available
velero backup get              # daily-full-backup-*、dr-course-backup-01 ...
```

順序錯了會怎樣

沒有先放 velero-repo-credentials → Velero 產生一組新密碼 → Data Mover 無法開啟既有的 kopia repository → 資源還原成功、PV 資料失敗。又是一個「看起來大致成功」的陷阱。

08

建立每日備份：備份整個叢集

單元 8

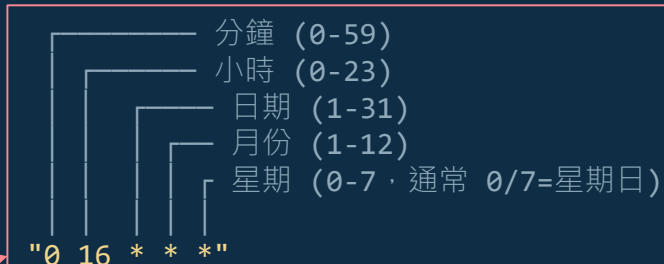
每日自動備份叢集資源與持久化資料，確保災難發生時具備還原能力



先看 YAML : Schedule = 時間 + Backup 範本

schedule-daily-full.yaml (建議改良版)

```
apiVersion: velero.io/v1
kind: Schedule
metadata:
  name: daily-full-backup
  namespace: velero
spec:
  schedule: "0 16 * * *" # UTC 01:00 = 台灣 09:00
  useOwnerReferencesInBackup: false
  paused: false
  template: # 每次產生的 Backup spec
    includedNamespaces: ["*"]
    excludedNamespaces:
      - kube-system # 另含 kube-public 等
      - velero # ⚠ 一定要排除
      - rook-ceph # 用 Rook 重建
    excludedResources:
      - events # 另含 VolumeSnapshot
    snapshotVolumes: true
    snapshotMoveData: true
    storageLocation: default
    ttl: 720h0m0s # 保留 30 天
```



schedule : "0 1 * * *"

cron 格式，以 Velero server 的 UTC 計算 → 台灣時間 00:00。也可以寫 CRON_TZ=Asia/Taipei 前綴。

template

就是每次要產生的 Backup spec，欄位跟單元 3 的 Backup 一樣；產生的備份叫 daily-full-backup-<UTC 時間戳>。

excludedNamespaces : velero

線上排程沒有排除 velero，每份備份夾帶其他備份的 DataUpload，同一個 PVC 有多筆結果 → PV 資料還原失敗。

兩個安全開關 useOwnerReferencesInBackup 保持 false：刪除 Schedule 時不會連帶刪除它產生的備份。paused: true 可以暫停排程但保留設定。

八個單元 + 補充，九句話

1 · 概念

etcd 只有描述，PV 裡才有內容。

2 · 外部儲存

備份不能跟被備份的東西一起死掉。

3 · Backup

Completed 之後，還有 7 項要檢查。

4 · Restore

成功的定義是災難前的資料查得到。

5 · etcd 策略

snapshot 只有一半，金鑰與 PKI 是另一半。

6 · etcd 還原

能不還原就不還原；要還原，3 台同步。

7 · 叢集重建

基礎元件自己裝，應用才用 Velero。

8 · 制度

沒有演練過的 RTO，只是一個願望。

補充 · 陷阱

Velero 不知道你的應用需要什麼。

Thank You

備份、復原與災難復原演練

補充

Backup 的陷阱：有備份但不一定救得回來

補充

補充單元（課後自學）：每一份備份都是 Completed、0 errors、0 warnings，但全部救不回一個可用的應用。



Lab 設計：一個 PostgreSQL 依賴三樣東西

依賴 1

PVC

資料

依賴 2

Secret

帳密

依賴 3

ConfigMap

初始化 SQL

少了任何一個，還原後的症狀都不一樣。另外還有一個使用 [rook-ceph-block-legacy](#) StorageClass 的 legacy-writer (情境 4、5)。

部署後寫入業務資料 (實測)

order_no	note
SEED-0001	initdb 種子資料
ORDER-1001	客戶 A 的訂單
ORDER-1002	客戶 B 的訂單
ORDER-1003	客戶 C 的訂單

(4 rows)

← ConfigMap 產生
← 只在 PVC 裡

```
kubectl apply -f velero/backup-pitfalls/manifest/  
kubectl -n dr-pitfall exec -i deploy/postgres -- psql -U shop -d shop < velero/backup-pitfalls/business-data.sql
```

四份範圍不同的備份，看起來一模一樣

```
velero backup create pitfall-full --include-namespaces dr-pitfall --snapshot-move-data
velero backup create pitfall-no-data --include-namespaces dr-pitfall --snapshot-volumes=false
velero backup create pitfall-no-secret ... --exclude-resources secrets --snapshot-move-data
velero backup create pitfall-no-configmap ... --exclude-resources configmaps --snapshot-move-data
```

```
$ velero backup get
```

NAME	STATUS	ERRORS	WARNINGS	CREATED
pitfall-full	Completed	0	0	2026-09-23 16:33:16 +0800 CST
pitfall-no-configmap	Completed	0	0	2026-09-23 16:35:01 +0800 CST
pitfall-no-data	Completed	0	0	2026-09-23 16:34:02 +0800 CST
pitfall-no-secret	Completed	0	0	2026-09-23 16:34:04 +0800 CST

Velero 不知道你的應用需要什麼

Velero 只負責「你要它備份的東西有沒有備份到」。範圍對不對只有應用負責人知道，唯一的驗證方法是還原。以下每個情境都用 namespaceMapping 還原到獨立 namespace。

情境 1：Resource 有了，但 Data 不存在

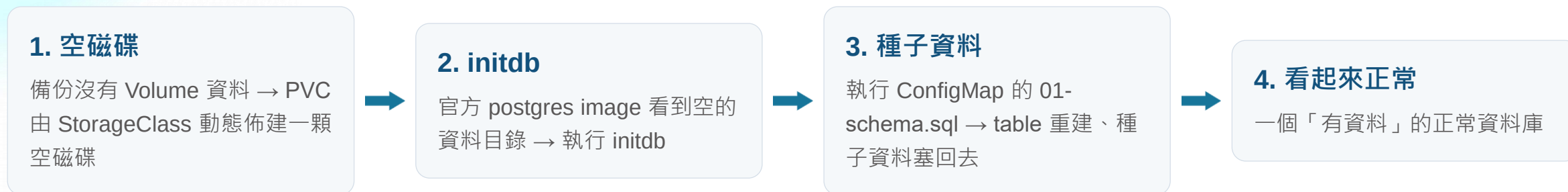
```
$ velero restore get pitfall-restore-no-data
STATUS      ERRORS  WARNINGS
Completed   0       2

$ kubectl get pods,pvc -n dr-pitfall-nodata
pod/postgres-5bf9b8b7cc-mqdnh 1/1 Running
persistentvolumeclaim/postgres-data Bound

$ kubectl -n dr-pitfall-nodata exec
deploy/postgres -- \
    psql -c 'SELECT order_no, note FROM orders;'
order_no |      note
-----+-----
SEED-0001 | initdb 種子資料
(1 row)
```

檢查	結果
Restore phase	✓ Completed
Pod	✓ Running 1/1
PVC	✓ Bound
DB 能連線、orders table 存在	✓ 通過
有資料	✓ 有一筆...
業務資料 ORDER-1001~1003	X 全部消失

情境 1：發生了什麼事



legacy-writer 也一樣：檔案時間戳記從 08:33:01 (原始) 變成 08:36:07 (還原後重新寫的)。

防範

驗證腳本要查「災難前才存在、不可能自動重生」的資料 (業務資料、時間戳記早於備份時間的紀錄)，不是查 table 在不在，也不是 `count(*) > 0`。